

Week_1__ROS2__Lab

February 28, 2026

1 Mobile Robots Lab

1.1 Week 1 Lab Manual: Linux Onboarding, ROS 2 Concepts, Workspace Setup, and First Package/Node

Course: MCT-454L Mobile Robotics Instructor: Dr. Maria Akram

ROS Distribution: Humble (recommended)

Lab Duration: 3 hours (suggested)

1.1.1 What you will produce by the end of this lab

1. A working ROS 2 development workspace (`~/ros2_ws`)
 2. A Python ROS 2 package (`my_first_pkg`)
 3. A runnable node (`simple_node`) executed using `ros2 run`
 4. Evidence of completion (screenshots + short answers)
-

2 0. Lab Rules and Submission

2.1 Submission checklist

Each student must maintain an individual GitHub repository for this course. After completing the lab, push all required files to your repository and submit the repository URL.

2.2 Repository Naming Convention

The repository must be named:

MR_Lab_YourFullName

Example:

MR_Lab_AliKhan

2.3 Required Repository Structure

Inside your repository, create a folder named:

week1/

Your structure must be exactly:

MR_Lab_YourName/ week0/ week1/ my_first_pkg/ screenshots/ answers.md
 README.md

2.4 Contents of week1 Folder

2.4.1 1. my_first_pkg/

- Complete ROS 2 package folder
- Must build successfully when cloned

2.4.2 2. screenshots/

Must include the following:

- ros2 version verification
- Successful colcon build
- ros2 pkg list showing your package
- ros2 run execution output

Screenshots must clearly show: - Your system username - Terminal commands used - Visible timestamps if possible

2.4.3 3. answers.md

Include written answers to all post-lab questions.

2.4.4 4. README.md

Must include: - Brief description of Week 1 lab - Commands used - Problems faced and how they were solved - Reflection (minimum 5–8 lines)

2.5 Git Commit Requirements

- Minimum one meaningful commit during the lab session
- Commit message must be descriptive

Examples:

Created ROS2 workspace

Added simple_node implementation

Registered entry point in setup.py

Completed Week 1 tasks

Repositories with a single last-minute commit may receive reduced marks.

2.6 Evaluation Criteria

Your submission will be evaluated based on:

- Correct repository structure
 - Successful ROS 2 package creation
 - Correct node execution
 - Quality of commit history
 - Clarity of documentation in README
 - Completeness of answers
-

2.7 Important Notes

- Repository must remain accessible until grading is complete.
 - Late commits are visible in Git history.
 - Copying another student's repository constitutes academic misconduct.
-

2.8 Academic integrity

- You may discuss concepts with classmates, but your code and answers must be your own.
 - Screenshots must show your machine username/terminal prompt to avoid copying.
-

3 1. Linux Primer for First-Time Users (Essential Only)

ROS 2 development involves frequent terminal usage. This section gives you the minimum you need to work confidently.

3.1 1.1 Terminology

- Terminal: a window where you type commands.
- Directory: a folder.
- Path: the location of a file/directory (for example: `/home/student/ros2_ws`).
- Home directory: your personal folder, represented by `~`.

3.2 1.2 Navigation commands

Run each command and observe the output:

```
pwd
ls
ls -l
cd ~
```

```
cd /
cd ~
mkdir -p ~/practice_linux/subfolder
cd ~/practice_linux
pwd
cd ..
pwd
```

3.3 1.3 Editing files

You can edit files using any editor. Recommended: - **nano** (beginner-friendly) - **gedit** (GUI, if available) - **code** (VS Code, if installed)

Examples:

```
nano test.txt
gedit test.txt
```

3.4 1.4 Common terminal shortcuts

- Tab auto-completes file/folder/command names
- Up Arrow shows previous commands
- Ctrl + C stops a running process
- Ctrl + L clears the screen (does not delete anything)

4 2. ROS 2 Conceptual Overview

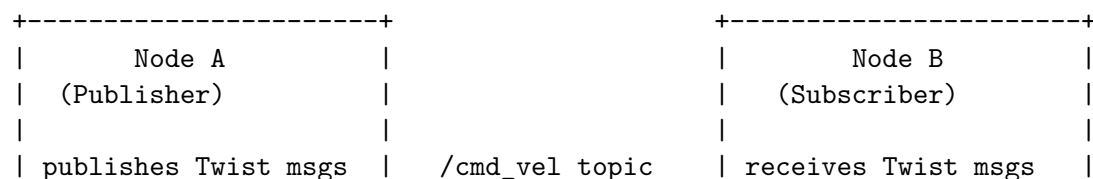
ROS 2 is a framework for building robot software using a distributed architecture.

4.1 2.1 Key vocabulary

- Node: a process that performs computation (example: controller, sensor driver, planner)
- Topic: a named communication channel for streaming messages
- Publisher: sends messages on a topic
- Subscriber: receives messages from a topic
- Service: request/response communication (like calling a function over the network)
- Action: long-running goal with feedback (used for navigation goals)
- Package: a folder with ROS 2 code, dependencies, and build information
- Workspace: a folder that contains one or more ROS 2 packages and build outputs

4.2 2.2 Conceptual diagram (topics)

Below is a conceptual diagram showing topic communication.



```
| at 10 Hz          +----->| and acts on them |
+-----+          +-----+
```

4.3 2.3 How ROS 2 runs your code

When you type: `ros2 run <package> <executable>`

ROS 2: 1. Finds the installed package in your sourced workspace 2. Locates the registered executable (entry point) 3. Runs it as a normal process 4. That process becomes a node when it calls `rclpy.init()` and constructs a `Node` object

5 3. Verify ROS 2 Installation and Environment

Open a new terminal.

5.1 3.1 Check ROS 2 availability

```
ros2 --version
```

If you see an error like `command not found`, your ROS 2 environment is not sourced.

5.2 3.2 Source ROS 2 (temporary for this terminal)

For ROS 2 Humble:

```
source /opt/ros/humble/setup.bash
```

Verify again:

```
ros2 --version
```

5.3 3.3 Confirm environment variables (optional)

```
printenv | grep -E "ROS|RMW|AMENT"
```

6 4. Create a ROS 2 Workspace

A workspace is a folder where you build and install packages.

6.1 4.1 Create the standard structure

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

6.2 4.2 First build (empty workspace)

```
colcon build
```

6.3 4.3 Source the workspace (so ROS 2 can find built packages)

```
source install/setup.bash
```

6.4 4.4 Make sourcing persistent (recommended)

This automatically sources your workspace when you open a new terminal:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

Then either open a new terminal or run:

```
source ~/.bashrc
```

6.5 4.5 Workspace folder meanings

After building, you will have:

- `src/` : your packages (source code)
- `build/` : build artifacts (intermediate files)
- `install/` : installed packages and executables
- `log/` : logs from the build process

Quick view:

```
cd ~/ros2_ws
ls
```

7 5. Create Your First Python Package

7.1 5.1 Create a package in the src folder

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python my_first_pkg
```

7.2 5.2 Rebuild and source

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

7.3 5.3 Confirm that ROS can see your package

```
ros2 pkg list | grep my_first_pkg
```

If nothing appears, see Troubleshooting (Section 9).

8 6. Understand the Package Structure (What each file does)

Inspect the package:

```
cd ~/ros2_ws/src/my_first_pkg
ls
```

Typical structure:

```
my_first_pkg/
  package.xml
  setup.py
  setup.cfg
  resource/
  my_first_pkg/
    __init__.py
```

8.1 6.1 package.xml

Declares package name, version, maintainer, and dependencies.

8.2 6.2 setup.py and setup.cfg

Define how Python code is installed, and registers runnable executables (console scripts).

8.3 6.3 resource/

Required for ROS indexing of packages.

8.4 6.4 my_first_pkg/ (Python module directory)

Contains your Python files (nodes).

9 7. Write a First Node (Python)

Goal: Create a node that prints a message when it starts.

9.1 7.1 Create the node file

Create: ~/ros2_ws/src/my_first_pkg/my_first_pkg/simple_node.py

You may use nano:

```
nano ~/ros2_ws/src/my_first_pkg/my_first_pkg/simple_node.py
```

Paste the following code in the editor.

```
[ ]: import rclpy
    from rclpy.node import Node
```

```

class SimpleNode(Node):
    def __init__(self):
        super().__init__('simple_node')
        self.get_logger().info('Hello, ROS 2')

def main(args=None):
    rclpy.init(args=args)
    node = SimpleNode()

    # spin_once lets us create the node, log once, and exit cleanly
    rclpy.spin_once(node, timeout_sec=0.1)

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

9.2 7.2 Make file executable (good practice)

```
chmod +x ~/ros2_ws/src/my_first_pkg/my_first_pkg/simple_node.py
```

10 8. Register the Node as an Executable (Entry Point)

ROS 2 needs to know what command name maps to which Python function.

10.1 8.1 Edit setup.py

Open:

```
nano ~/ros2_ws/src/my_first_pkg/setup.py
```

Find the `entry_points` section and make it look like this (only the `console_scripts` part matters):

```

entry_points={
    'console_scripts': [
        'simple_node = my_first_pkg.simple_node:main',
    ],
},

```

Save the file.

10.2 8.2 Rebuild and source

```

cd ~/ros2_ws
colcon build

```



```
source install/setup.bash
```

11 9. Run the Node and Verify Execution

Run:

```
ros2 run my_first_pkg simple_node
```

Expected output (similar):

```
[INFO] [simple_node]: Hello, ROS 2
```

11.1 9.1 Verify the node list (optional demonstration)

For long-running nodes, you would typically do:

```
ros2 node list
```

Our node exits quickly, so it may not appear. In Week 2, you will create nodes that keep running and you will inspect them using `ros2 node list` and `ros2 topic list`.

12 10. Troubleshooting (Read This Before Asking for Help)

12.1 Problem A: `ros2: command not found`

Cause: ROS 2 is not sourced in the terminal. Fix:

```
source /opt/ros/humble/setup.bash
```

Then retry:

```
ros2 --version
```

12.2 Problem B: `colcon: command not found`

Cause: `colcon` not installed or not in `PATH`. Fix (Ubuntu):

```
sudo apt update
```

```
sudo apt install python3-colcon-common-extensions
```

12.3 Problem C: Package '`my_first_pkg`' not found

Common causes: 1. Workspace not sourced 2. Build failed 3. Package created in the wrong folder

Fix steps:

```
cd ~/ros2_ws
```

```
colcon build
```

```
source install/setup.bash
```

```
ros2 pkg list | grep my_first_pkg
```

Also confirm you created it inside `~/ros2_ws/src`.

12.4 Problem D: `ros2 run my_first_pkg simple_node` says executable not found

Cause: entry point not registered correctly or build not re-run. Fix: 1. Re-check `setup.py` `console_scripts` line 2. Rebuild and source:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

12.5 Problem E: Python error (`SyntaxError`, `ModuleNotFoundError`)

Cause: code typo or missing dependency. Fix: - Open the file and check indentation and quotes carefully - Ensure `import rclpy` works (ROS environment sourced) - Rebuild after changes

12.6 Problem F: You edited files but changes do not take effect

Cause: You forgot to rebuild and/or source. Fix:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

12.7 How to ask for help (required format)

When you ask for help, include: 1. The exact command you ran 2. The full error text (copy/paste) 3. Your folder structure output:

```
cd ~/ros2_ws
find src/my_first_pkg -maxdepth 2 -type f
```

13 11. Experiment Tasks (Graded)

13.1 Task 1: Customize the log message

Change the node to print: `Welcome to Mobile Robotics Lab`

Rebuild and run again. Include screenshot.

13.2 Task 2: Add a counter

Modify the node to print a counter value each time it runs, like: `Run count: 1` Then increment to 2, 3, etc.

Hint: Store the counter in a text file in your package directory and read/update it each run.

13.3 Task 3 (Advanced): Print your name using a ROS parameter

Add a parameter called `student_name` and print it. Expected behavior: - If parameter is set: print it - If not set: print `student_name not set`

You will learn parameters in more depth later, but try this challenge now.

14 12. Post-Lab Questions (Answer in answers.md)

1. Define: node, topic, package, workspace. Provide one sentence each.
 2. Explain why sourcing is required. What happens if you do not source a workspace?
 3. What is the purpose of `colcon build`? What folders does it generate?
 4. In your own words, explain what the `entry_points` console script does in `setup.py`.
 5. Draw (by hand or ASCII) a diagram showing one publisher and one subscriber connected by a topic.
-

15 GitHub Submission Guide

After completing Week 1:

1. Create folder `week1` inside your repository.
2. Copy `my_first_pkg` into `week1/`.
3. Add screenshots folder.
4. Add `answers.md`.
5. Commit and push:

```
git add .
git commit -m "Week 1 completed"
git push
```

Required Structure:

```
AMR_Lab_YourName/  week0/  week1/  my_first_pkg/  screenshots/  answers.md
README.md
```